

FSDP2 (Fully Sharded Data Parallel) - PyTorch Code Location & Architecture

Generated: March 22, 2026

Source: PyTorch /torch/distributed/fsdp/_fully_shard/ directory

CONVERSATION SERIES: Complete Technical Deep-Dive into FSDP2 Implementation

PART 1: FSDP CODE LOCATION - KEY LOGIC OVERVIEW

DIRECTORY STRUCTURE:

/Users/shizhefu0/Desktop/ml/code/public_github/pytorch/torch/distributed/fsdp/_fully_shard/

KEY FILES AND THEIR PURPOSES:

1. _fully_shard.py (1026 lines)

PURPOSE: Main FSDP2 public API and FSDPModule class

KEY COMPONENTS:

- fully_shard() decorator (lines 87-274): Entry point applying FSDP to modules
- FSDPModule class (lines 291-1026): User-facing API

METHODS IN FSDPModule:

- reshard(): Reshards parameters, frees unsharded ones
- unshard(async_op=False): Allocates and all-gathers parameters
- set_is_last_backward()
- set_requires_gradient_sync()
- set_requires_all_reduce()
- set_reshard_after_forward()
- set_reshard_after_backward()
- set_modules_to_forward_prefetch()
- set_modules_to_backward_prefetch()
- set_custom_all_gather()
- set_custom_reduce_scatter()
- set_all_reduce_hook()
- set_post_optim_event()
- set_gradient_divide_factor()
- set_force_sum_reduction_for_comms()
- set_unshard_in_backward()
- set_allocate_memory_from_process_group_for_comm()
- set_symm_mem_for_comm()

2. _fsdp_state.py

PURPOSE: Module-level FSDP state and control flow coordination

KEY METHODS:

- `_pre_forward()` (lines 268-299): Module-level forward preparation
- `_post_forward()` (lines 301-323): Module-level cleanup after forward
- `_pre_backward()` (lines 329-341): Autograd hook entry point for backward
- `_root_post_backward_final_callback()`: Finalizes backward across param groups

DESIGN PATTERN:

State-level methods orchestrate parameter-group-level execution.

Each method prepares the module and delegates to parameter groups.

3. `_fsdp_param_group.py`

PURPOSE: Parameter group-level communication and state management

KEY METHODS:

- `pre_forward()` (lines 473-482): Issues all-gather, waits, registers hooks
- `post_forward()` (lines 484-492): Reshards if configured
- `pre_backward()` (lines 503-514): All-gathers if not already done
- `post_backward()` (lines 516-600+): Accumulates gradients, reduce-scatters

DESIGN PATTERN:

Parameter-group methods execute the actual communication and computation coordination at the fine-grained level.

4. `_fsdp_collectives.py`

PURPOSE: Implementation of all-gather and reduce-scatter communication

KEY FUNCTIONS:

- `foreach_all_gather()` (lines 325-397): Two-stage all-gather collective
- `foreach_reduce()` (lines 524-678): Reduce-scatter for gradients
- `all_gather_comm()`: Orchestrates collective communication
- `reduce_scatter_comm()`: Orchestrates reduce-scatter communication

DESIGN PATTERN:

Collectives module encapsulates communication implementation details.

Supports custom communication backends via `AllGather` and `ReduceScatter` APIs.

5. `_fsdp_init.py`

PURPOSE: FSDP initialization and dynamic class injection

KEY FUNCTIONS:

- `_apply_to_module()` (lines 404-427): Dynamic class creation mechanism
- `_init_param_group()`: Initializes parameter groups
- `_get_modules_and_states()`: Extracts module hierarchy

DESIGN PATTERN:

Dynamic class injection allows FSDP to wrap arbitrary modules without requiring explicit inheritance.

6. `_fsdp_param.py`

PURPOSE: Parameter-level wrapper and metadata

KEY CLASS: `FSDPParam`

DESIGN PATTERN:

Each parameter gets wrapped with sharding metadata (DTensor on dim-0).

```
=====
=====
PART 2: DYNAMIC CLASS INJECTION - _apply_to_module() EXPLAINED
=====
=====
```

QUESTION ADDRESSED:

"Please explain this code: _apply_to_module()"

LOCATION: torch/distributed/fsdp/_fully_shard/_fsdp_init.py, lines 404-427

FUNCTION SIGNATURE:

```
-----
def _apply_to_module(
    modules: list[nn.Module],
    cls_to_fsdps: dict[type, type],
    fsdp_module_class: type,
    class_name_suffix: str,
    deepcopy_fn: Callable[[Any], Any],
):
```

PURPOSE:

Dynamically creates and injects wrapper classes that inherit from both the FSDP functionality class and the original module class. This makes FSDP available to arbitrary user modules without requiring explicit inheritance.

MECHANISM:

1. DYNAMIC CLASS CREATION:

For each unique original class in the module hierarchy, creates a new class with dual inheritance:

```
class FSDP<OriginalClassName>(FSDPModule, OriginalModule):
    pass
```

2. MRO (Method Resolution Order):

[FSDP<Orig>, FSDPModule, Orig, ...]

This ensures:

- FSDP methods have highest priority (checked first)
- FSDPModule provides FSDP functionality
- Original class is next in MRO for fallback
- Index 2 points to original class

3. CLASS CACHING:

New classes are cached in cls_to_fsdps dictionary to avoid recreating the same class multiple times

4. MODULE REBINDING:

Each module instance is rebound to the new dynamically created class

while preserving its internal state.

EXAMPLE FLOW:

Input Module:

```
class MyModel(nn.Module):
    def __init__(self):
        self.layer = nn.Linear(10, 10)
```

After `fully_shard(model)`:

Dynamically created class:

```
class FSDPMyModel(FSDPModule, MyModel):
    pass
```

model instance now has type: `FSDPMyModel`

model can now call: `reshard()`, `unshard()`, `set_requires_gradient_sync()`, etc.

KEY INSIGHT:

This design is crucial because:

- User modules don't need to inherit from `FSDPModule`
- Multiple different module types can be FSDP-wrapped independently
- Each wrapped module gets full FSDP functionality without code modification
- Clean separation: FSDP logic is orthogonal to module logic

=====

=====

PART 3: FORWARD PASS FLOW - ALL-GATHER → COMPUTE → RESHARD

=====

=====

QUESTION ADDRESSED:

"Please explain Forward: All-gather parameters → compute → optionally reshard by using exact code"

HIGH-LEVEL FLOW:

Module Forward Flow:

1. Pre-forward hook triggered (registered in `_pre_forward`)
2. All-gather parameters across distributed workers
3. Wait for all-gather to complete
4. Execute module's forward computation
5. Post-forward hook triggered
6. Optionally reshard (squeeze back to sharded form)

DETAILED STEP-BY-STEP:

STEP 1: PRE-FORWARD ORCHESTRATION (`_fsdp_state.py`, lines 268-299)

=====

Method: `_pre_forward(module, args, kwargs)`

Responsibilities:

- Prepare module for forward
- Coordinate with all parameter groups
- Update state machine (STATE_IDLE -> STATE_FORWARD)
- Handle mixed precision casting if needed

Pseudo-code flow:

```
for each param_group:
    param_group.unshard(async_op=False)
    # Waits for all-gather to complete
```

STEP 2: PARAMETER-GROUP LEVEL ALL-GATHER (lines 473-482)

=====

Method: `pre_forward()` in `FSDPPParamGroup`

Code flow:

1. Check if parameters already unsharded
 - If yes, skip all-gather (already in forward)
2. Issue all-gather collective:

```
all_gather_result = foreach_all_gather(
    fsdp_params=self.fsdps_params,
    group=self.all_gather_group,
    async_op=False, # Wait immediately
    all_gather_copy_in_stream=stream1,
    all_gather_stream=stream2,
    device=device,
    all_gather_comm=self._all_gather_comm,
)
```
3. Wait for all-gather in `all_gather_stream`
4. Register backward hook for `pre_backward`

STEP 3: COLLECTIVE ALL-GATHER OPERATION (lines 325-397)

=====

Function: `foreach_all_gather()` in `_fsdp_collectives.py`

Two-stage all-gather process:

Stage 1: Copy-in (Prepare data for communication)

- Input: Sharded parameters on device
- Operation: `torch.ops.fsdps.all_gather_copy_in()`
- Output: `all_gather_input` buffer
- Buffer size: `all_gather_input_numel` (not multiplied)

Stage 2: Collective Communication

- Input: `all_gather_input` buffer

- Operation: `all_gather_comm()`
 - Allocates output buffer: `size = input_numel * world_size`
 - Performs collective all-gather
 - All workers gather all parameters
- Output: `all_gather_output` buffer (`size = input_numel * world_size`)

Stage 3: Copy-out (Finalize parameters)

- Input: `all_gather_output` buffer
- Operation: Register output in `param.unsharded_data`
- Result: Parameters now unsharded and registered

Memory Layout (All-gather):

Before:

```
Worker 0: [P0 chunk]
Worker 1: [P1 chunk]
Worker 2: [P2 chunk]
Worker 3: [P3 chunk]
```

After all-gather:

```
Worker 0: [P0 chunk, P1 chunk, P2 chunk, P3 chunk]
Worker 1: [P0 chunk, P1 chunk, P2 chunk, P3 chunk]
Worker 2: [P0 chunk, P1 chunk, P2 chunk, P3 chunk]
Worker 3: [P0 chunk, P1 chunk, P2 chunk, P3 chunk]
```

STEP 4: FORWARD COMPUTATION

The module's `forward()` method now executes with full unsharded parameters. Computation can happen on all gathered parameters without resharding in between.

STEP 5: POST-FORWARD RESHARDING (`_fsdp_state.py`, lines 301-323)

Method: `_post_forward(module, args, output)`

Decision point: Check `reshard_after_forward` flag

If `reshard_after_forward == True`:

- Call `reshard()` to squeeze parameters back to sharded form
- Free unsharded parameters
- Register sharded parameters to module
- Save memory for backward pass

Pseudocode:

```
for param_group in param_groups:
    param_group.reshard()
    # Resharding process:
    # 1. Reshape unsharded parameter to original shape
    # 2. Shard along dim-0
    # 3. Create DTensor with Shard(0) placement
    # 4. Register as module parameter
```

If `reshard_after_forward == False`:

- Keep unsharded parameters in memory
- Avoid all-gather in backward
- Trade higher memory for reduced communication
- Default for root module

STREAM MANAGEMENT:

- `all_gather_copy_in_stream`: Overlaps copy-in with computation
- `all_gather_stream`: Waits before forward computation starts
- Synchronization: `all_gather_stream.wait_stream(all_gather_copy_in_stream)`

KEY DESIGN DECISIONS:

1. Two-stage all-gather (copy-in + collective) allows overlap
2. Reshard is optional - trades memory vs communication
3. State-level (module) vs param-group-level separation
 - State handles orchestration
 - Param groups handle execution
4. Hook-based integration with `nn.Module`
 - Pre-forward and post-forward hooks manage FSDP ops

=====

=====

PART 4: BACKWARD PASS FLOW - ALL-GATHER → GRADIENTS → REDUCE-SCATTER

=====

=====

QUESTION ADDRESSED:

"Please explain Backward: All-gather parameters → compute gradients → reduce-scatter gradients by using exact code"

HIGH-LEVEL FLOW:

Backward Flow:

1. Pre-backward hook triggered (registered in `pre_forward`)
2. All-gather parameters (if resharded after forward)
3. Autograd computes gradients
4. Post-backward hook triggered
5. Reduce-scatter accumulated gradients
6. Free unsharded parameters

DETAILED STEP-BY-STEP:

STEP 1: PRE-BACKWARD ENTRY (`_fsdp_state.py`, lines 329-341)

=====

Method: `_pre_backward(module)`

Context: Autograd registered this as a backward hook during `pre_forward`

Responsibilities:

- Entry point for backward pass
- Coordinate with parameter groups
- Check if parameters need all-gathering

Pseudo-code:

```
for fsdp_param_group in state._fsdp_param_groups:  
    fsdp_param_group.pre_backward()
```

STEP 2: PARAMETER-GROUP LEVEL BACKWARD PREP (lines 503-514)

=====

Method: `pre_backward()` in `FSDPParamGroup`

Code flow:

1. Check if parameters were resharded after forward
 - If yes: need to re-all-gather before gradients computed
2. Issue all-gather collective:

```
all_gather_result = foreach_all_gather(  
    fsdp_params=self.fsdp_params,  
    group=self.all_gather_group,  
    async_op=False,  
    all_gather_copy_in_stream=stream1,  
    all_gather_stream=stream2,  
    device=device,  
    all_gather_comm=self._all_gather_comm,  
)
```
3. Wait for all-gather to complete
 - Now parameters ready for gradient computation
4. Register `post_backward` hook

STEP 3: GRADIENT COMPUTATION (AUTOGRAD)

=====

PyTorch's autograd backward pass computes gradients with respect to unsharded parameters. This is transparent to FSDP.

STEP 4: POST-BACKWARD REDUCTION (lines 516-600+)

=====

Method: `post_backward()` in `FSDPParamGroup`

Complex process with multiple stages:

Stage 1: Gradient Accumulation

- Gradients computed by autograd
- Accumulated in `param.grad`

Stage 2: Reshape/Rechunk for Reduction

- Input: Unsharded gradients (shape = original shape)
- Operation: Reshape to match forward all-gather layout
- Purpose: Prepare for reduce-scatter operation

Example:

Original param shape: (100, 100)
world_size: 4

After reshape for reduce-scatter:

Shape: (25, 100) on each worker ($100 / 4 = 25$)

But conceptually in reduce-scatter layout:

Worker 0: gradient chunk [0:25, :]
Worker 1: gradient chunk [25:50, :]
Worker 2: gradient chunk [50:75, :]
Worker 3: gradient chunk [75:100, :]

Stage 3: Reduce-scatter Collective

Function: `foreach_reduce()` in `_fsdp_collectives.py` (lines 524-678)

Process:

1. Allocate `reduce_scatter_input` buffer
Size: `total_gradient_numel`
2. Copy-in gradients to input buffer
Operation: `torch.ops.fsdps.reduce_scatter_copy_in()`
3. Perform reduce-scatter collective
 - All workers send their full gradient
 - Result: Each worker gets reduced gradient for its shard
 - Collective operation:
`reduce_scatter_comm(
 reduce_scatter_input,
 reduce_scatter_output,
 group=reduce_scatter_group,
)`
4. Output buffer allocation
Size: `total_gradient_numel / world_size`

Example output:

Worker 0 receives: reduced gradient [0:25, :]
Worker 1 receives: reduced gradient [25:50, :]
Worker 2 receives: reduced gradient [50:75, :]
Worker 3 receives: reduced gradient [75:100, :]

Memory Layout (Reduce-scatter):

Before:

Worker 0: `full_grad [0:100, :]`
Worker 1: `full_grad [0:100, :]`
Worker 2: `full_grad [0:100, :]`
Worker 3: `full_grad [0:100, :]`

After reduce-scatter:

Worker 0: `reduced_grad [0:25, :]` (sum of all workers' [0:25, :])

Worker 1: reduced_grad [25:50, :] (sum of all workers' [25:50, :])
Worker 2: reduced_grad [50:75, :] (sum of all workers' [50:75, :])
Worker 3: reduced_grad [75:100, :] (sum of all workers' [75:100, :])

Stage 4: Reshard for Optimizer

- After reduce-scatter: gradients are sharded by dim-0
- Create DTensor with Shard(0) placement
- Result: Gradients match parameter sharding for optimizer step

Stage 5: Free Unsharded Parameters

- Unsharded parameters from backward are freed
- Only sharded parameters remain for optimizer

Pseudo-code:

```
for param in fsdp_params:
    param.unsharded_data = None
    param.data = param.sharded_param # DTensor with Shard(0)
```

ARCHITECTURAL LAYERS:

1. Module Level (_fsdp_state.py)
 - _pre_backward(): Coordinates backward across param groups
 - _root_post_backward_final_callback(): Finalizes backward
2. Param Group Level (_fsdp_param_group.py)
 - pre_backward(): Prepares group for backward
 - post_backward(): Manages gradient reduction
3. Collective Level (_fsdp_collectives.py)
 - foreach_reduce(): Executes reduce-scatter communication
 - Handles device placement, dtype, stream management

STREAM MANAGEMENT:

- all_gather_copy_in_stream: Used in pre_backward for all-gather
- all_gather_stream: Used in pre_backward for all-gather
- reduce_scatter_stream: Used in post_backward for reduce-scatter
- Synchronization ensures dependencies between stages

KEY DESIGN INSIGHTS:

1. Pre-backward all-gathers only if resharded after forward
 - Saves communication if reshard_after_forward=False
2. Reduce-scatter reduces all gradients efficiently
 - Each worker keeps only its shard
3. State machine tracks backward progression
 - Prevents incorrect hook firing
4. Gradient division factor support
 - Allows custom reduce operations (e.g., NCCL PreMulSum)

```
=====
=====
PART 5: METHOD COMPARISON - PRE-FORWARD VS FORWARD HOOKS
=====
=====
```

QUESTION ADDRESSED:

"What are the differences between '@_dynamo_disable def _pre_forward' and 'def pre_forward'?"

LAYER 1: MODULE-LEVEL _pre_forward()

```
=====
```

Location: _fsdp_state.py, lines 268-299

Decorator: @_dynamo_disable

Scope: One per FSDPState (entire module tree)

Trigger: Module's forward pre-hook

Responsibilities:

- Orchestrate all parameter groups
- Update state machine (IDLE -> FORWARD)
- Handle mixed precision
- Coordinate with all groups before any computation

Pseudocode:

```
@_dynamo_disable
def _pre_forward(self, module, args, kwargs):
    # State machine update
    self._state_ctx.state = TrainingState.FORWARD

    # Prepare all parameter groups
    for param_group in self._fsdp_param_groups:
        param_group.lazy_init()
        param_group.unshard(async_op=False)
        param_group.wait_for_unshard()

    # Mixed precision handling if configured
    if self._mp_policy.param_dtype is not None:
        # Cast parameters to computation dtype

    return args, kwargs
```

LAYER 2: PARAMETER-GROUP LEVEL pre_forward()

```
=====
```

Location: _fsdp_param_group.py, lines 473-482

Decorator: None

Scope: One per FSDPParamGroup (group of parameters)

Trigger: Called from _pre_forward state method

Responsibilities:

- Issue all-gather collective for specific param group
- Wait for all-gather completion
- Register backward hook for this group

Pseudocode:

```
def pre_forward(self):
    if self.params_already_unsharded():
        return # Skip if already unsharded from before

    # Issue all-gather
    all_gather_result = foreach_all_gather(
        fsdp_params=self.fsdp_params,
        group=self.all_gather_group,
        async_op=False,
        all_gather_copy_in_stream=stream1,
        all_gather_stream=stream2,
        device=self.device,
        all_gather_comm=self._all_gather_comm,
    )

    # Wait for all-gather
    # (Implicit in stream synchronization)

    # Record forward order for backward
    self._forward_order = self._current_iteration

    # Register backward hook
    self._register_pre_backward_hook()
```

KEY DIFFERENCES:

=====

Aspect	_pre_forward()	pre_forward()
Scope	Entire module tree	Single parameter group
Called by	nn.Module hook system	State's _pre_forward method
Frequency	Once per module	Once per parameter group
Responsibilities	Orchestration	Execution of specific group
State machine	Updates global state	No state updates
Mixed precision	Handles casting	Assumes dtype already correct
All-gather	Coordinates all groups	Issues for specific group
Backward hook reg	None (delegates)	Registers pre_backward hook
Decorator	@_dynamo_disable (TorchScript-incompatible code)	
	None (no special requirements)	

EXECUTION SEQUENCE:

=====

1. Module.forward() called
2. Module's pre-forward hook fires
 - > Calls _pre_forward()
3. _pre_forward() calls:

```

for param_group in param_groups:
    param_group.unshard(async_op=False)
    -> Internally calls pre_forward()
    -> Issues all-gather for this group
    -> Waits for completion
    -> Registers backward hook
4. All groups now have unsharded parameters
5. Original forward() method executes

```

DESIGN RATIONALE:

=====

Two-layer separation enables:

1. Module-level coordination
 - Ensures all groups are ready before forward
 - Handles global state transitions
 - Mixed precision decisions apply uniformly
2. Group-level execution
 - Fine-grained control per group
 - Allows independent prefetching strategies
 - Can be used standalone if needed

=====

PART 6: METHOD COMPARISON - PRE-BACKWARD VS POST-BACKWARD

=====

QUESTION ADDRESSED:

"What are the differences between '@_dynamo_disable def _pre_backward' and '@_dynamo_disable def pre_backward'?"

LAYER 1: MODULE-LEVEL _pre_backward()

=====

Location: _fsdp_state.py, lines 329-341

Decorator: @_dynamo_disable

Scope: One per FSDPState (entire module tree)

Trigger: Autograd backward hook registered in pre_forward

Responsibilities:

- Entry point for backward pass
- State machine update (FORWARD -> PRE_BACKWARD)
- Coordinate all parameter groups for backward prep
- Ensure all groups are ready for gradient computation

Pseudocode:

```

@dynamo_disable
def _pre_backward(self, module):
    # Update state machine

```

```
self._state_ctx.state = TrainingState.PRE_BACKWARD
```

```
# Prepare all parameter groups for backward
for param_group in self._fsdp_param_groups:
    param_group.pre_backward()
```

LAYER 2: PARAMETER-GROUP LEVEL pre_backward()

=====

Location: `_fsdp_param_group.py`, lines 503-514

Decorator: None

Scope: One per FSDPParamGroup

Trigger: Called from `_pre_backward` state method

Responsibilities:

- Issue all-gather if needed (if resharded after forward)
- Wait for all-gather completion
- Ensure parameters ready for gradient computation
- Register `post_backward` hook

Pseudocode:

```
def pre_backward(self):
    # Check if we need to re-all-gather
    if self.reshard_after_forward:
        # Parameters were resharded, need to unshard again
        all_gather_result = foreach_all_gather(
            fsdp_params=self.fsdparams,
            group=self.all_gather_group,
            async_op=False,
            all_gather_copy_in_stream=stream1,
            all_gather_stream=stream2,
            device=self.device,
            all_gather_comm=self._all_gather_comm,
        )
        # Wait for result

    # Register post_backward hook
    self._register_post_backward_hook()
```

LAYER 3: PARAMETER-GROUP LEVEL post_backward()

=====

Location: `_fsdp_param_group.py`, lines 516-600+

Decorator: `@_dynamo_disable` (some parts)

Scope: One per FSDPParamGroup

Trigger: Autograd backward hook registered in `pre_backward`

Responsibilities:

- Accumulate gradients
- Reduce-scatter gradients across workers
- Reshard gradients to match parameter sharding
- Free unsharded parameters

- Handle mixed precision gradient computation

Multi-stage process:

Stage 1: Gradient accumulation

- Input: Autograd computed gradients
- Check gradient state (computed but not reduced)

Stage 2: Reshape for reduce-scatter

- Prepare gradient tensor for reduce-scatter layout
- Match the forward all-gather chunking

Stage 3: Reduce-scatter operation

```
all_gather_result = foreach_reduce(
    fsdp_params=self.fsdp_params,
    unsharded_grads=accumulated_grads,
    reduce_scatter_group=self.reduce_scatter_group,
    reduce_scatter_stream=stream,
    reduce_scatter_comm=self._reduce_scatter_comm,
)
```

Result: Each worker has reduced gradient for its parameter shard

Stage 4: Gradient resharding

- Reshape reduced gradients to parameter shape
- Create DTensor with Shard(0) placement
- Register sharded gradients to parameters

Stage 5: Cleanup

- Free unsharded parameter data
- Free unsharded gradient data
- Update state for next iteration

EXECUTION SEQUENCE IN BACKWARD:

=====

1. Autograd backward starts
2. Pre-backward hook fires (registered in pre_forward)
 - > Calls `_pre_backward()`
3. `_pre_backward()` calls:
 - for `param_group` in `param_groups`:
 - `param_group.pre_backward()`
 - > Re-all-gathers if resharded after forward
 - > Registers `post_backward` hook
4. Autograd computes gradients
5. Post-backward hook fires (registered in `pre_backward`)
 - > Calls `post_backward()`
 - > Reduce-scatters gradients
 - > Frees unsharded memory

KEY DIFFERENCES:

=====

Aspect	<code>_pre_backward()</code>	<code>pre_backward()</code>	<code>post_backward()</code>
Scope	Entire module tree	Single param group	Single param group
Called by	Autograd hook	<code>_pre_backward()</code>	Autograd hook
Frequency	Once per backward	Once per backward	Once per backward
Timing	Start of backward	Start of backward	After gradients computed
Responsibilities	Orchestration	Prep for gradients	Reduce & finalize grads
State machine update	PRE_BACKWARD state	None	Might update to POST_BACKWARD
All-gather	Coordinates	Issues if needed	None
Reduce-scatter	None	None	Issues reduce-scatter
Gradient handling	None	None	Reduces & reshards

MEMORY MANAGEMENT PATTERN:

=====

Forward:

`pre_forward()`: All-gather -> unsharded params in memory

`forward()`: Compute

`post_forward()`: Reshard -> free unsharded, keep sharded

Backward:

`pre_backward()`: All-gather (if resharded) -> unsharded params

`backward()`: Compute gradients

`post_backward()`: Reduce-scatter -> free unsharded, keep reduced

DESIGN RATIONALE:

=====

Separation into three methods enables:

1. Module-level coordination (`_pre_backward`)
 - State machine update happens once
 - All groups synchronized
2. Group-level prep (`pre_backward`)
 - Conditional all-gather only if needed
 - Allows group-specific strategies
3. Group-level finalization (`post_backward`)
 - Gradient reduction happens independently per group
 - Can overlap reduce-scatter across groups

KEY INSIGHT: ASYMMETRY

=====

Forward pass:

One all-gather operation at the start

Backward pass:

- Conditional re-all-gather (only if resharded)
- Reduce-scatter at the end
- More complex due to gradient reduction requirement

PART 7: COMMUNICATION PRIMITIVES - ALL-GATHER & REDUCE-SCATTER

QUESTION ADDRESSED:

"Please explain this Communication (all-gather, reduce-scatter) using the code here"

OVERVIEW:

FSDP uses two main collective communication operations:

1. ALL-GATHER

- When: Pre-forward (and pre-backward if resharded)
- What: Gather all parameter shards from all workers
- Why: Need full parameters for forward computation
- Result: Each worker has complete unsharded parameters

2. REDUCE-SCATTER

- When: Post-backward
- What: Reduce gradients and scatter results
- Why: Each worker keeps only its gradient shard
- Result: Each worker has reduced gradient for its shard

PART A: ALL-GATHER IMPLEMENTATION

Function: `foreach_all_gather()` in `_fsdp_collectives.py`, lines 325-397
 Location: `torch/distributed/fsdp/_fully_shard/_fsdp_collectives.py`

FUNCTION SIGNATURE:

```
def foreach_all_gather(
    fsdp_params: list[FSDPParam],
    group: dist.ProcessGroup,
    async_op: bool,
    all_gather_copy_in_stream: torch.cuda.Stream,
    all_gather_stream: torch.cuda.Stream,
    device: torch.device,
    all_gather_comm: AllGather,
) -> AllGatherResult:
    """
    All-gathers the unsharded parameters for the list of FSDPParams.
```

Args:

fsdp_params: List of FSDP-wrapped parameters
group: Distributed process group for communication
async_op: Whether to use async all-gather
all_gather_copy_in_stream: Stream for copy-in operations
all_gather_stream: Stream for collective communication
device: Device to place parameters on
all_gather_comm: Communication backend for all-gather

Returns:

AllGatherResult with completion information

"""

THREE-STAGE ALL-GATHER PROCESS:

=====

STAGE 1: COPY-IN OPERATION (Prepare data)

Location: Within `foreach_all_gather`, starting ~line 340

Purpose: Move sharded parameters to communication buffer

Operation:

```
output = torch.ops.fsdps.all_gather_copy_in(  
    inputs=all_gather_inputs, # List of sharded params  
    output_buffer=output,     # Pre-allocated buffer  
    stream=all_gather_copy_in_stream,  
)
```

Process:

1. Take sharded parameters from each worker
2. Copy into communication buffer
3. Buffer layout: parameter chunks concatenated
4. Buffer size: `total_param_numel` (not multiplied by `world_size` yet)

Buffer allocation:

```
all_gather_input_numel = sum(param.numel() for param in fsdp_params)  
output = torch.empty(  
    all_gather_input_numel,  
    dtype=param_dtype,  
    device=device,  
)
```

Example (2 parameters, 4 workers):

Input shards:

Worker 0: `param1_shard[0]` (size 100), `param2_shard[0]` (size 50)
Worker 1: `param1_shard[1]` (size 100), `param2_shard[1]` (size 50)
Worker 2: `param1_shard[2]` (size 100), `param2_shard[2]` (size 50)
Worker 3: `param1_shard[3]` (size 100), `param2_shard[3]` (size 50)

Copy-in output (each worker):

`[param1_shard, param2_shard]` = [100 values, 50 values] = 150 values total

Buffer size: 150 (called all_gather_input_numel)

STAGE 2: COLLECTIVE COMMUNICATION (All-gather)

Location: Within foreach_all_gather, ~line 350

Purpose: Distribute shards from all workers to all workers

Operation:

```
all_gather_comm(  
    input_tensor=all_gather_input_buffer,    # Size: input_numel  
    output_tensor=all_gather_output_buffer,  # Size: input_numel * world_size  
    group=group,  
    async_op=async_op,  
)
```

Buffer allocation:

```
all_gather_output = all_gather_comm.allocate(  
    shard_size=all_gather_input_numel,  
    device=device,  
    dtype=param_dtype,  
)  
# Returns buffer of size: all_gather_input_numel * world_size
```

Process (NCCL all-gather):

1. Send all_gather_input from each worker
2. Receive all_gather_inputs from all other workers
3. Each worker now has: [input_w0, input_w1, input_w2, input_w3]
4. Output size: input_numel * world_size (complete tensor)

Example memory layout:

Before all-gather:

Worker 0 buffer:	[P1_0, P2_0]	(size 150)
Worker 1 buffer:	[P1_1, P2_1]	(size 150)
Worker 2 buffer:	[P1_2, P2_2]	(size 150)
Worker 3 buffer:	[P1_3, P2_3]	(size 150)

After all-gather (each worker):

Worker 0 output:	[P1_0, P2_0, P1_1, P2_1, P1_2, P2_2, P1_3, P2_3]	(size 600)
Worker 1 output:	[P1_0, P2_0, P1_1, P2_1, P1_2, P2_2, P1_3, P2_3]	(size 600)
Worker 2 output:	[P1_0, P2_0, P1_1, P2_1, P1_2, P2_2, P1_3, P2_3]	(size 600)
Worker 3 output:	[P1_0, P2_0, P1_1, P2_1, P1_2, P2_2, P1_3, P2_3]	(size 600)

STAGE 3: COPY-OUT OPERATION (Finalize parameters)

Location: After collective communication, ~line 365

Purpose: Register gathered parameters to module

Operation:

```

for i, fsdp_param in enumerate(fsdp_params):
    # Extract parameter chunk from all-gather output
    param_start = param_offsets[i]
    param_end = param_offsets[i] + param_numel[i]
    unsharded_param = all_gather_output[param_start:param_end]

    # Register to param
    fsdp_param.unsharded_data = unsharded_param.reshape(param_shape)

```

Result:

- Parameters are contiguous unsharded tensors
- Available for forward computation
- Can be used with original computation graph

STREAM SYNCHRONIZATION:

Two independent streams manage overlap:

`all_gather_copy_in_stream` (stream 1):

- Runs copy-in operations
- Can overlap with forward computation

`all_gather_stream` (stream 2):

- Runs collective communication
- Waits on copy-in to complete:
 - `all_gather_stream.wait_stream(all_gather_copy_in_stream)`
- Computation waits on this stream before forward

Synchronization:

- `current_stream.wait_stream(all_gather_stream)`
 - Main computation waits for all-gather to complete

KEY PARAMETERS:

Name	Purpose
fsdp_params	Parameters to all-gather
group	Process group for collective
async_op	True = non-blocking, False = blocking
all_gather_copy_in_stream	Stream for pre-communication operations
all_gather_stream	Stream for collective communication
device	Computation device (usually GPU)
all_gather_comm	Custom comm backend (default NCCL)

PART B: REDUCE-SCATTER IMPLEMENTATION

=====

Function: `foreach_reduce()` in `_fsdp_collectives.py`, lines 524-678

Location: `torch/distributed/fsdp/_fully_shard/_fsdp_collectives.py`

FUNCTION SIGNATURE:

```
def foreach_reduce(
    fsdp_params: list[FSDPParam],
    unsharded_grads: dict[FSDPParam, torch.Tensor],
    reduce_scatter_group: dist.ProcessGroup,
    reduce_scatter_stream: torch.cuda.Stream,
    reduce_scatter_comm: ReduceScatter,
    device: torch.device,
    gradient_divide_factor: float = 1.0,
    # ... other args
) -> None:
    """
    Reduce-scatters the unsharded gradients across workers.

    Args:
        fsdp_params: List of parameters with accumulated gradients
        unsharded_grads: Mapping of parameter to its gradient
        reduce_scatter_group: Process group for communication
        reduce_scatter_stream: Stream for collective
        reduce_scatter_comm: Communication backend
        device: Device placement
        gradient_divide_factor: Factor to divide gradients by
    """
```

FIVE-STAGE REDUCE-SCATTER PROCESS:

=====

STAGE 1: PREPARE PARAMETERS FOR REDUCTION

Location: Start of foreach_reduce, ~line 530

Purpose: Check gradient state and prepare data

Process:

```
for fsdp_param in fsdp_params:
    # Verify gradient exists
    gradient = unsharded_grads[fsdp_param]

    # Check gradient state
    if not gradient.is_cuda:
        raise ValueError("Gradients must be on CUDA device")
```

STAGE 2: RESHAPE/RECHUNK FOR REDUCE-SCATTER

Location: Gradient preparation section, ~line 540

Purpose: Reshape gradients to match reduce-scatter chunking

Original gradient shape: (original tensor shape)

All-gather layout shape: (original tensor shape)
Reduce-scatter input layout: (numel,) but conceptually:
- Dimension 0 divided by world_size
- Flattened for communication

Reshaping process:
world_size = dist.get_world_size(reduce_scatter_group)
rank = dist.get_rank(reduce_scatter_group)

```
original_size = gradient.numel()  
shard_size = original_size // world_size
```

```
# Rechunk gradient for reduce-scatter  
# Dimension 0 becomes shard dimension
```

Example (parameter shape: [100, 50], 4 workers):

Original:
shape = [100, 50], numel = 5000

Rechunked for reduce-scatter:
shape = [25, 50], numel = 1250 (100 / 4 = 25)
Actually flattened: [1250]

Reduce-scatter chunks:
Worker 0: [0:1250]
Worker 1: [1250:2500]
Worker 2: [2500:3750]
Worker 3: [3750:5000]

STAGE 3: ALLOCATE REDUCE-SCATTER BUFFERS

Location: Buffer allocation section, ~line 555

Input buffer (before reduce-scatter):

```
reduce_scatter_input = reduce_scatter_comm.allocate(  
    full_size=total_gradient_numel,  
    shard_size=total_gradient_numel // world_size,  
    dtype=gradient_dtype,  
    device=device,  
)
```

Size calculation:
total_gradient_numel = sum(g.numel() for g in all_gradients)
input_buffer_size = total_gradient_numel (not divided yet)

Output buffer (after reduce-scatter):

```
reduce_scatter_output = reduce_scatter_comm.allocate(  
    full_size=total_gradient_numel,  
    shard_size=total_gradient_numel // world_size,
```

```
dtype=gradient_dtype,  
device=device,  
)
```

Size calculation:

output_buffer_size = total_gradient_numel // world_size

Example: 5000 numel / 4 workers = 1250 per worker

STAGE 4: COPY-IN AND REDUCE-SCATTER COLLECTIVE

Location: Communication section, ~line 560

Copy-in operation:

```
output = torch.ops.fsdpr.reduce_scatter_copy_in(  
    inputs=unsharded_gradients, # List of gradient tensors  
    input_buffer=reduce_scatter_input,  
    stream=reduce_scatter_stream,  
)
```

Purpose: Organize gradients into communication buffer

Layout:

input_buffer = [grad1_full, grad2_full, ...]

Size: sum of all gradient numel

Collective reduce-scatter:

```
reduce_scatter_comm(  
    input_tensor=reduce_scatter_input,    # Size: total_numel  
    output_tensor=reduce_scatter_output,  # Size: total_numel / world_size  
    group=reduce_scatter_group,  
    async_op=False,  
)
```

Operations:

- Each worker sends full gradient
- Reduction operation: SUM (average if dividing by factor)
- Each worker receives its shard of reduced gradient

Memory layout during reduce-scatter:

Before collective (all workers have full gradients):

Worker 0:

Input buffer: [G1_full, G2_full] = 5000 numel

Worker 1:

Input buffer: [G1_full, G2_full] = 5000 numel

Worker 2:

Input buffer: [G1_full, G2_full] = 5000 numel

Worker 3:

Input buffer: [G1_full, G2_full] = 5000 numel

After collective (each worker has its shard of reduced):

Worker 0:

Output buffer: [reduced_G1[0:1250], reduced_G2[0:???]] = 1250 numel

Worker 1:

Output buffer: [reduced_G1[1250:2500], reduced_G2[???:???]] = 1250 numel

Worker 2:

Output buffer: [reduced_G1[2500:3750], ...] = 1250 numel

Worker 3:

Output buffer: [reduced_G1[3750:5000], ...] = 1250 numel

STAGE 5: GRADIENT DIVISION AND RESHARDING

Location: Post-reduction section, ~line 570

Gradient division (if specified):

if gradient_divide_factor != 1.0:

 reduce_scatter_output /= gradient_divide_factor

Purpose: Average gradients across workers if needed

Method: Can use NCCL PreMulSum for zero-copy division

Reshape to parameter shape:

```
for i, fsdp_param in enumerate(fsdp_params):
```

```
    # Extract from reduce-scatter output
```

```
    grad_chunk = reduce_scatter_output[grad_offsets[i]:grad_offsets[i+1]]
```

```
    # Reshape to parameter shape
```

```
    sharded_grad = grad_chunk.reshape(param_shape_on_this_worker)
```

```
    # Create DTensor with Shard(0) placement
```

```
    sharded_grad = DTensor.from_local_tensor(
```

```
        sharded_grad,
```

```
        mesh=fsdp_param.mesh,
```

```
        placements=[Shard(0)],
```

```
    )
```

```
    # Register to parameter
```

```
    fsdp_param.grad = sharded_grad
```

Stream management:

reduce_scatter_stream runs asynchronously
 Main stream waits before optimizer step:
 current_stream.wait_stream(reduce_scatter_stream)

KEY PARAMETERS:

Name	Purpose
fsdp_params	Parameters with gradients
unsharded_grads	Dictionary of param -> gradient
reduce_scatter_group	Process group for communication
reduce_scatter_stream	Stream for collective
reduce_scatter_comm	Communication backend
device	GPU device
gradient_divide_factor	Division factor (for averaging)

COLLECTIVE OPERATION COMPARISON:

Aspect	All-Gather	Reduce-Scatter
Input size	N (per rank)	N (per rank)
Output size	N * world_size	N / world_size
Operation	Gather	Reduce + Scatter
Buffers	Input + Output	Input + Output
Copy-in stage	Yes	Yes
Stream mgmt	Two streams	One stream
Gradient div	N/A	Optional
Typical use	Forward prep	Backward finalize

MEMORY EFFICIENCY:

All-gather:

Peak memory = 2 * param_size * world_size
 (original sharded + full unsharded during all-gather)

Reduce-scatter:

Peak memory = 2 * param_size
 (full gradient input + sharded output, not 1 + world_size)

SCALABILITY CONSIDERATIONS:

1. Communication time scales with world size
 - All-gather: $O(\text{world_size} * \text{param_size})$
 - Reduce-scatter: $O(\text{world_size} * \text{param_size})$

2. Overlapping computation with communication
 - Multiple streams allow pipelining
 - Copy-in can overlap with forward computation
 - Reduce-scatter can overlap with optimizer
3. Custom backends
 - AllGather and ReduceScatter APIs support custom implementations
 - Can optimize for specific hardware (e.g., NCCL, gloo)

=====

=====

SUMMARY: ARCHITECTURAL PATTERNS IN FSDP2

=====

=====

KEY DESIGN PATTERNS:

=====

1. TWO-LAYER ARCHITECTURE (State vs ParamGroup)
 - State level: Orchestration, coordination, global decisions
 - ParamGroup level: Execution, fine-grained control
 - Enables flexibility and modularity
2. HOOK-BASED INTEGRATION
 - Pre-forward hooks for all-gather
 - Post-forward hooks for optional reshard
 - Pre-backward hooks for all-gather (if needed)
 - Post-backward hooks for reduce-scatter
 - Clean integration with nn.Module
3. TWO-STREAM DESIGN FOR OVERLAP
 - Copy-in stream (compute-like operations)
 - Communication stream (collective operations)
 - Enables overlap of CPU operations and collective
4. DYNAMIC CLASS INJECTION
 - Wraps arbitrary modules without inheritance
 - Preserves module semantics
 - Clean separation of concerns
5. MULTI-STAGE COLLECTIVES
 - Copy-in: Prepare data for communication
 - Collective: Actual distributed operation
 - Copy-out: Finalize for computation
 - Allows customization at each stage
6. OPTIONAL RESHAPE AFTER FORWARD
 - reshard_after_forward=True: Save memory
 - reshard_after_forward=False: Save communication
 - reshard_after_forward=int: Hybrid approach
 - Flexible memory/communication tradeoff

EXECUTION FLOW SUMMARY:

=====

FORWARD PASS:

1. Module forward called
2. Pre-forward hook: all-gather parameters
3. Forward computation with unsharded parameters
4. Post-forward hook: optionally reshard (free unsharded)
5. Forward complete

BACKWARD PASS:

1. Autograd backward starts
2. Pre-backward hook: re-all-gather if resharded
3. Autograd computes gradients
4. Post-backward hook: reduce-scatter gradients
5. Optimizer step on sharded gradients
6. Backward complete

MEMORY USAGE:

Peak forward: $\text{param_size} * \text{world_size}$ (if not resharding)
Peak backward: $\text{param_size} * \text{world_size}$ (during all-gather)
Minimum: param_size (only sharded parameters)

COMMUNICATION VOLUME:

=====

Per iteration:

Forward: $\text{world_size} * \text{param_size}$ bytes (all-gather)
Backward: $\text{world_size} * \text{param_size}$ bytes (reduce-scatter)
Total: $2 * \text{world_size} * \text{param_size}$ bytes per iteration

Optimizations:

- Reshard after forward reduces backward all-gather
- Prefetching overlaps communication with computation
- Mixed precision reduces communication size

STATE MACHINE:

=====

States:

IDLE -> FORWARD -> (POST_FORWARD) ->
PRE_BACKWARD -> POST_BACKWARD -> IDLE

Hooks:

Pre-forward: IDLE -> FORWARD
Pre-backward: Forward -> PRE_BACKWARD
Post-backward: PRE_BACKWARD -> POST_BACKWARD (or IDLE)

CUSTOMIZATION POINTS:

=====

1. AllGather communication
 `set_custom_all_gather(comm: AllGather)`
2. ReduceScatter communication
 `set_custom_reduce_scatter(comm: ReduceScatter)`
3. Gradient reduction
 `set_all_reduce_hook(hook: Callable)`
 `set_gradient_divide_factor(factor: float)`
4. Memory allocation
 `set_allocate_memory_from_process_group_for_comm(enable: bool)`
 `set_symm_mem_for_comm(backend: str)`
5. Prefetching strategies
 `set_modules_to_forward_prefetch(modules: list)`
 `set_modules_to_backward_prefetch(modules: list)`
6. Resharding policies
 `set_reshard_after_forward(reshard: bool)`
 `set_reshard_after_backward(reshard: bool)`

=====
=====

END OF DOCUMENT

=====
=====

File generated with complete FSDP2 deep-dive conversation summary.
All code locations, implementations, and architectural patterns documented.

Contact: PyTorch FSDP2 Composable API Documentation
Date: March 22, 2026